ENSA Agadir – SITCN

Cybersécurité – Portfolio de projets

Projet 1 : CryptoVault

Bibliothèque de cryptographie en Python

Compte rendu d'avancement

Réalisé par : Abdellah SALMI

Dépôt GitHub : `https://github.com/secboy-star/cryptovault`

Date : 30 octobre 2026

Projet éducatif – ne pas utiliser en production.

Table des matières

1 Introduction **3**
1.1 Contexte . . . 3
1.2 Objectifs d'apprentissage . . . 3
1.3 État d'avancement . . . 3
1.4 Méthode de travail . . . 3
1.5 Transparence sur l'usage de l'intelligence artificielle . . . 3

2 Environnement et outils **3**
2.1 Outils utilisés . . . 4
2.2 Bibliothèques installées . . . 4
2.3 Configuration de Git et clonage du dépôt . . . 4
2.4 Environnement virtuel et dépendances . . . 4
2.5 Structure du projet . . . 5
2.6 Sécurité du dépôt : `.gitignore` . . . 6
2.7 Premier commit . . . 7

3 Niveau 0 – Initiation **7**
3.1 Étape 1 : Chiffre de César . . . 7
3.1.1 Principe . . . 7
3.1.2 Implémentation (`src/classical.py`) . . . 7
3.1.3 Résultats . . . 8
3.1.4 Bug découvert : les lettres accentuées . . . 9
3.1.5 Limites . . . 10
3.2 Étape 2 : Chiffre de Vigenère . . . 10
3.2.1 Principe . . . 10
3.2.2 Implémentation . . . 10
3.2.3 Résultats . . . 11
3.2.4 Bug découvert : la clé `key1` . . . 12
3.2.5 Limites . . . 13
3.3 Étape 3 : Chiffrement XOR . . . 13
3.3.1 Principe . . . 14
3.3.2 Implémentation . . . 14
3.3.3 Résultats . . . 14
3.3.4 Attaque à texte clair connu . . . 14
3.3.5 Malléabilité : aucun contrôle d'intégrité . . . 14
3.3.6 Conclusion . . . 15
3.4 Étape 4 : AES-256-GCM . . . 15
3.4.1 Principe . . . 15
3.4.2 Implémentation (`src/symmetric.py`) . . . 15
3.4.3 Résultats . . . 16
3.4.4 Comparaison XOR / AES-GCM . . . 18
3.4.5 Bonnes pratiques . . . 18

4 Historique des commits **19**

5 Difficultés rencontrées et leçons **19**

6 Limites et améliorations prévues **20**

7 Conclusion et suite **20**

Références **20**

1 Introduction

1.1 Contexte

CryptoVault est le premier projet d'une trajectoire de projets en cybersécurité. Il consiste à construire, pas à pas, une bibliothèque de cryptographie en Python : des chiffrements classiques (César, Vigenère, XOR) jusqu'à un gestionnaire de secrets avec PKI, messagerie chiffrée de bout en bout et rotation de clés.

1.2 Objectifs d'apprentissage

— Comprendre les fondements mathématiques de la cryptographie.

— Implémenter des algorithmes symétriques (AES-GCM) et asymétriques (RSA-OAEP).

— Construire une infrastructure PKI avec certificats X.509.

— Développer un gestionnaire de secrets sécurisé avec journal d'audit.

1.3 État d'avancement

Niveau / Étape	Contenu	Statut
Niveau 0 – Étape 1	Chiffre de César	Terminé
Niveau 0 – Étape 2	Chiffre de Vigenère	Terminé
Niveau 0 – Étape 3	Chiffrement XOR	Terminé
Niveau 0 – Étape 4	AES-256-GCM	Terminé
Niveau 0 – Étape 5	RSA (chiffrement et signature)	En cours
Niveau 0 – Étape 6	Échange de clés Diffie-Hellman	À venir
Niveaux 1, 2, 3	Bibliothèque, PKI et messagerie E2E, gestionnaire de secrets	À venir

Table 1 – Avancement du projet à la date de ce compte rendu.

1.4 Méthode de travail

Pour chaque étape, la même démarche est appliquée :

1. écrire le code à la main et comprendre chaque ligne ;
2. le tester, puis essayer de le casser (entrées limites, clés invalides, données modifiées) ;
3. enregistrer le résultat par une capture d'écran ;
4. créer un *commit* Git au message explicite, puis le publier sur GitHub ;
5. consigner dans ce compte rendu le principe, le résultat, les erreurs rencontrées et les limites.

1.5 Transparence sur l'usage de l'intelligence artificielle

Un assistant d'IA (Claude, développé par Anthropic) est utilisé comme *tuteur* : il explique le code ligne par ligne et propose des pistes d'expérimentation. Le code est saisi, exécuté, testé et modifié par l'étudiant, et les erreurs découvertes lors des tests (sections 3.1.4 et 3.2.4) sont analysées dans ce document.

2 Environnement et outils

2.1 Outils utilisés

Outil	Détail
Système	Windows
Langage	Python 3.13.5
Éditeur	Visual Studio Code
Gestion de versions	Git et GitHub (dépôt public)
Environnement virtuel	`venv` (dossier `crypto_env`)

Table 2 – Outils de développement.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python --version
Python 3.13.5
```

Figure 1 – Vérification de la version de Python installée (`python -version`).

2.2 Bibliothèques installées

Les dépendances sont figées dans `requirements.txt` via `pip freeze`. Les bibliothèques principales sont :

Bibliothèque	Version	Rôle
pycryptodome	4.0.0	AES-GCM, RSA, hachages
cryptography	50.0.2	Diffie-Hellman, certificats X.509
Flask	3.1.3	Messagerie chiffrée (niveau 2)
bcrypt	5.0.0	Hachage de mots de passe (niveau 1)

Table 3 – Bibliothèques principales (versions issues de `requirements.txt`).

2.3 Configuration de Git et clonage du dépôt

Après la configuration de l'identité Git (`user.name` et `user.email`), le dépôt créé sur GitHub (avec README, `.gitignore` Python et licence MIT) est cloné en local.

```
C:\Users\HP\Desktop\New folder>git config --global user.name "Mr.Salmi"

C:\Users\HP\Desktop\New folder>git config --global user.email "abdellahsalmi147@gmail.com"

C:\Users\HP\Desktop\New folder>git clone https://github.com/secboy-star/cryptovault.git
Cloning into 'cryptovault'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
```

Figure 2 – Configuration de Git et clonage du dépôt `cryptovault`.

2.4 Environnement virtuel et dépendances

Un environnement virtuel isole les bibliothèques du projet de celles du système. Il est créé et activé, puis les dépendances sont installées.

```
python -m venv crypto_env
crypto_env\Scripts\activate
pip install pycryptodome cryptography flask bcrypt
pip freeze > requirements.txt
```

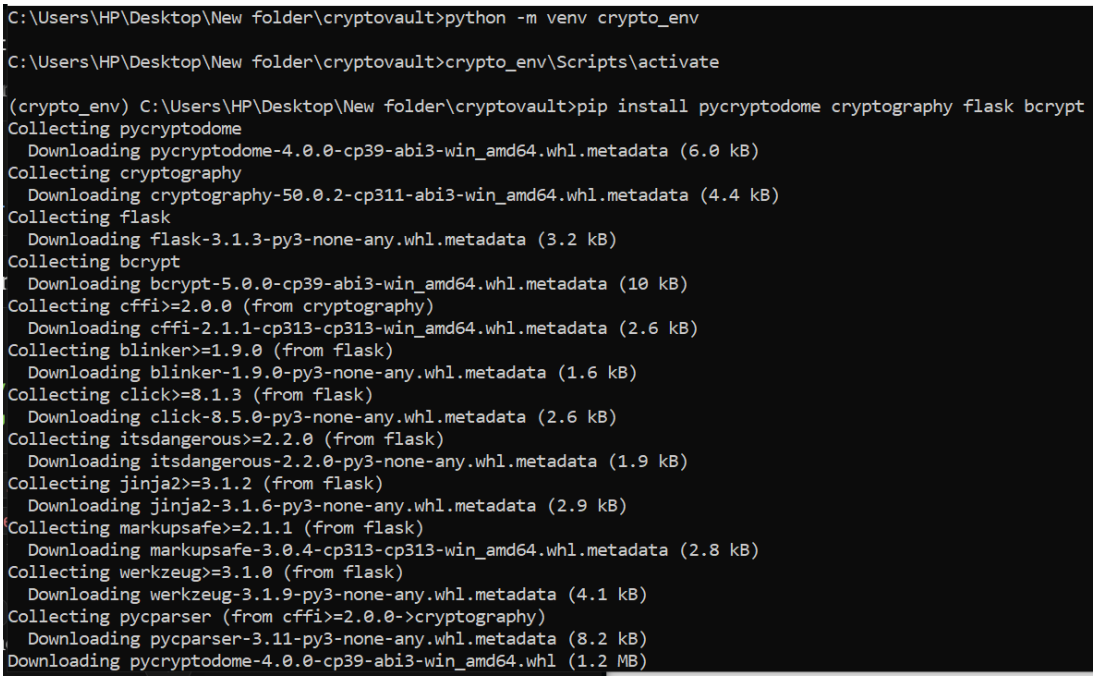


Figure 3 – Création de l'environnement virtuel et installation des dépendances.

Le fichier `requirements.txt` est ensuite généré :

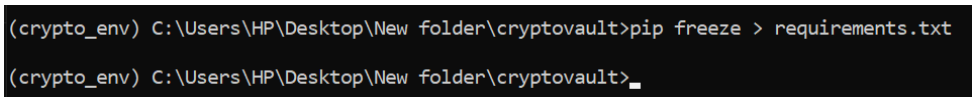


Figure 4 – Génération de `requirements.txt` avec `pip freeze`.

2.5 Structure du projet

```
cryptovault/
|-- src/
|   |-- __init__.py
|   |-- classical.py      # Cesar, Vigenere, XOR
|   |-- symmetric.py      # AES-GCM
|   |-- asymmetric.py     # RSA, Diffie-Hellman
|   |-- hashing.py        # SHA-256, bcrypt, HMAC
|   |-- pki.py            # Certificats X.509
|   |-- secrets_mgr.py    # Gestionnaire de secrets
|-- tests/
|-- messenger/            # Messagerie E2E
|-- docs/                 # Compte rendu, captures d'ecran
|-- requirements.txt
|-- README.md
```

Le fichier `__init__.py` indique à Python que `src` est un paquet importable ; c'est ce qui permet de lancer un module par `python -m src.classical`.

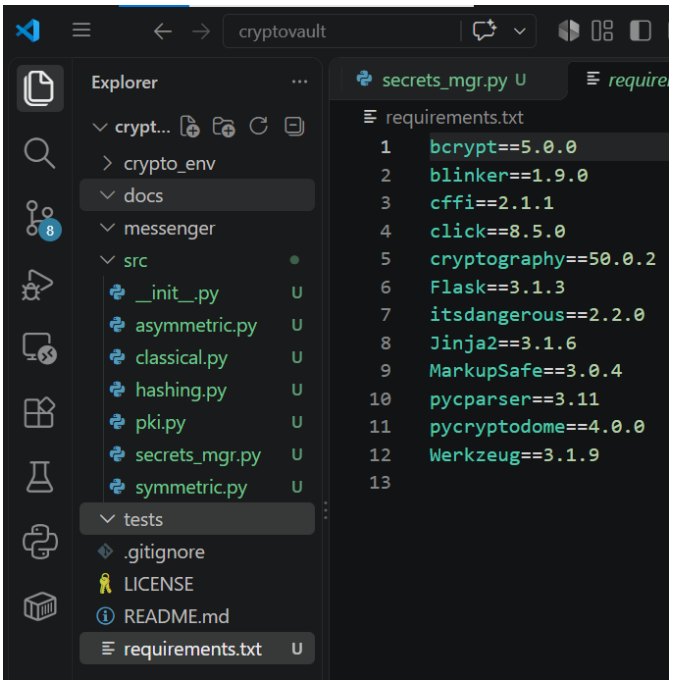


Figure 5 – Arborescence du projet et contenu de `requirements.txt` dans VS Code.

2.6 Sécurité du dépôt : `.gitignore`

Dans un projet de cryptographie, une clé privée publiée par erreur est une faute grave. Les lignes suivantes sont donc ajoutées à `.gitignore` avant le premier commit :

```
crypto_env/
*.pem
vault.enc
```

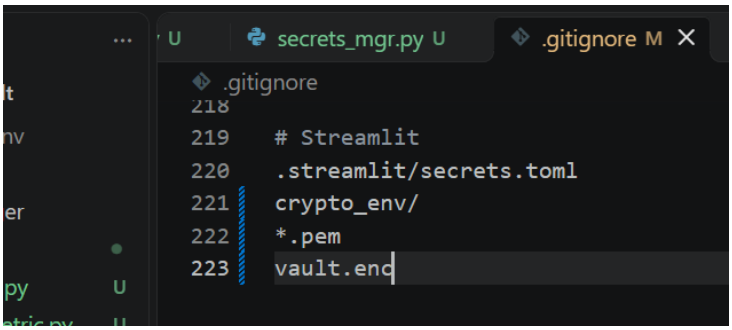


Figure 6 – Ajout des règles de sécurité dans `.gitignore`.

2.7 Premier commit

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git add .

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git commit -m "chore: setup project structure and dependencies"
[main 3784701] chore: setup project structure and dependencies
 9 files changed, 15 insertions(+)
 create mode 100644 requirements.txt
 create mode 100644 src/__init__.py
 create mode 100644 src/asymmetric.py
 create mode 100644 src/classical.py
 create mode 100644 src/hashing.py
 create mode 100644 src/pki.py
 create mode 100644 src/secrets_mgr.py
 create mode 100644 src/symmetric.py

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 730 bytes | 182.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/secboy-star/cryptovault.git
   1c65f72..3784701  main -> main

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>
```

Figure 7 – Premier commit `chore: setup project structure and dependencies` publié sur GitHub.

3 Niveau 0 – Initiation

3.1 Étape 1 : Chiffre de César

3.1.1 Principe

Chaque lettre est décalée d'un nombre fixe k de positions dans l'alphabet. En numérotant les lettres de 0 à 25 :

$$E_k(x) = (x + k) \bmod 26, \qquad D_k(y) = (y - k) \bmod 26.$$

Le déchiffrement est le chiffrement avec le décalage opposé $(-k)$: en Python, l'opérateur `%` renvoie un résultat positif, ce qui rend cette astuce valable.

3.1.2 Implémentation (`src/classical.py`)

```python
import string


def caesar_encrypt(plaintext, shift):
    result = []
    for char in plaintext:
        if char in string.ascii_letters:
            base = ord("A") if char.isupper() else ord("a")
            result.append(chr((ord(char) - base + shift) % 26 + base))
        else:
            result.append(char)
    return "".join(result)


def caesar_decrypt(ciphertext, shift):
    return caesar_encrypt(ciphertext, -shift)


def caesar_brute_force(ciphertext):
```

```
    for shift in range(26):
        print(f"Shift {shift:2d}: {caesar_decrypt(ciphertext, shift)}")
```

Exemple de calcul. Pour la lettre `S` avec $k = 13$: ord(S) $= 83$, $83 - 65 = 18$, $18 + 13 = 31$, $31 \bmod 26 = 5$, $5 + 65 = 70$ soit la lettre `F`.

3.1.3 Résultats

Premier test avec le message `SECURITE INFORMATION` et un décalage de 13 : le chiffré est `FRPHEVGR VASBEZNGVBA` et le déchiffrement retrouve le message d'origine.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Original: SECURITE INFORMATION
Chiffre: FRPHEVGR VASBEZNGVBA
Dechiffre: SECURITE INFORMATION
```

Figure 8 – Chiffrement et déchiffrement de `SECURITE INFORMATION` (décalage 13).

Attaque par force brute. La fonction `caesar_brute_force` essaie les 26 décalages possibles ; le texte lisible apparaît au décalage 13.

```
Shift  0: FRPHEVGR VASBEZNGVBA
Shift  1: EQOGDUFQ UZRADYMFUAZ
Shift  2: DPNFCTEP TYQZCXLETZY
Shift  3: COMEBSDO SXPYBWKDSYX
Shift  4: BNLDARCN RWOXAVJCRXW
Shift  5: AMKCZQBM QVNWZUIBQWV
Shift  6: ZLJBYPAL PUMVYTHAPVU
Shift  7: YKIAXOZK OTLUXSGZOUT
Shift  8: XJHZWNYJ NSKTWRFYNTS
Shift  9: WIGYVMXI MRJSVQEXMSR
Shift 10: VHFXULWH LQIRUPDWLRQ
Shift 11: UGEWTKVG KPHQTOCVKQP
Shift 12: TFDVSJUF JOGPSNBUJPO
Shift 13: SECURITE INFORMATION <--
Shift 14: RDBTQHSD HMENQLZSHNM
Shift 15: QCASPGRC GLDMPKYRGML
Shift 16: PBZROFQB FKCLOJXQFLK
Shift 17: OAYQNEPA EJBKNIWPEKJ
Shift 18: NZXPMDOZ DIAJMHVODJI
Shift 19: MYWOLCNY CHZILGUNCIH
Shift 20: LXVNKBMX BGYHKFTMBHG
Shift 21: KWUMJALW AFXGJESLAGF
Shift 22: JVTLIZKV ZEWFIDRKZFE
Shift 23: IUSKHYJU YDVEHCQJYED
Shift 24: HTRJGXIT XCUDGBPIXDC
Shift 25: GSQIFWHS WBTCFAOHWCB

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>
```

Figure 9 – Les 26 décalages possibles : le message clair apparaît pour le décalage 13.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git add src/classical.py

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>commit -m "feat: add caesar cipher (encrypt, decrypt, brute force)"
'commit' is not recognized as an internal or external command,
operable program or batch file.

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git commit -m "feat: add caesar cipher (encrypt, decrypt, brute force)"
[main b19ca29] feat: add caesar cipher (encrypt, decrypt, brute force)
 1 file changed, 24 insertions(+)

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 708 bytes | 354.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/secboy-star/cryptovault.git
   3784701..b19ca29  main -> main
```

Figure 10 – Commit `feat: add caesar cipher (encrypt, decrypt, brute force)` (hash `b19ca29`).

3.1.4 Bug découvert : les lettres accentuées

Avec la première version (condition `char.isalpha()`), le test sur `"Hellow Wörld! 2026"` donne un déchiffrement incorrect :

```
if __name__ == "__main__":
    message = "Hellow Wörld! 2026"
    encrypted = caesar_encrypt(message, 13)
    print (f"Original: {message}")
    print (f"Chiffre: {encrypted}")
    print (f"Dechiffre: {caesar_decrypt(enc
```

Figure 11 – Code du test : message contenant la lettre « ö ».

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Original: Hellow Wörld! 2026
Chiffre: Uryybj Jgeyq! 2026
Dechiffre: Hellow Wtrld! 2026
```

Figure 12 – Le « ö » est déchiffré en « t » : l'information est perdue.

Analyse. La méthode `isalpha()` accepte les lettres Unicode comme « ö », mais la formule suppose 26 lettres ASCII. Pour « ö » (code 246, base 97) :

$$(246 - 97 + 13) \bmod 26 = 6 \;\Rightarrow\; \texttt{g}.$$

Or la lettre « t » (code 116) donne aussi $(116 - 97 + 13) \bmod 26 = 6$, donc `g`. Deux lettres différentes ont le même chiffré : l'application n'est pas injective, le déchiffrement ne peut plus retrouver l'original.

Correction. La condition devient `char in string.ascii_letters` : seuls les 52 caractères A–Z et a–z sont chiffrés, les autres sont conservés tels quels. Après correction, le même test donne un chiffré où le « ö » reste inchangé, et le déchiffrement retrouve exactement le message d'origine :

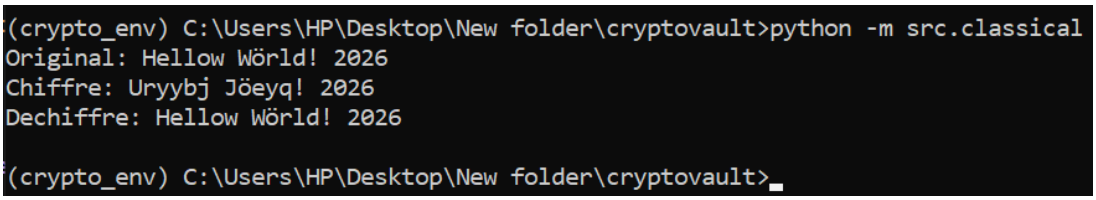


Figure 13 – Après correction : `Hellow Wörld! 2026` est correctement déchiffré.

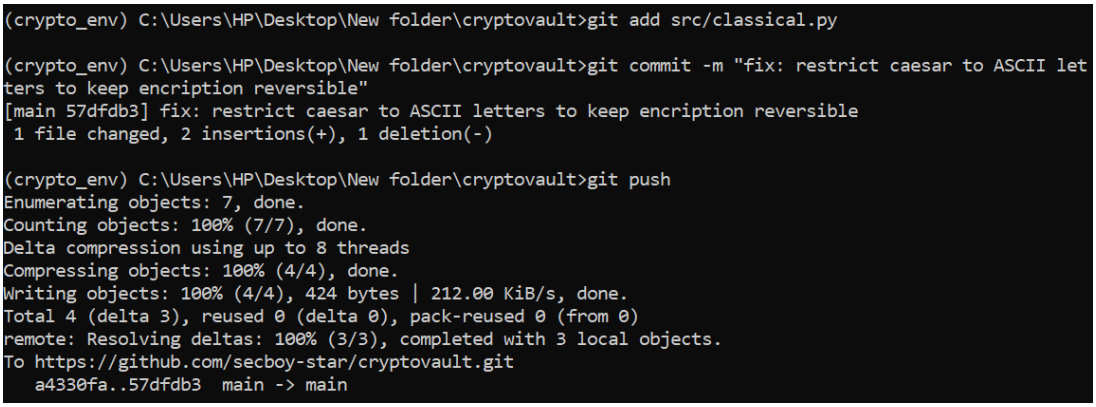


Figure 14 – Commit de correction `fix: restrict caesar to ASCII letters`.

3.1.5 Limites

— Seulement 26 clés possibles : une attaque par force brute (`caesar_brute_force`) casse le chiffre instantanément.

— Les lettres accentuées ne sont plus chiffrées (donc visibles en clair). Amélioration possible : normaliser le texte avec `unicodedata` avant chiffrement.

3.2 Étape 2 : Chiffre de Vigenère

3.2.1 Principe

Chiffrement polyalphabétique : la clé est un mot, répété sur le message. Chaque lettre de la clé fournit un décalage (`A` $= 0$, `B` $= 1$, …). Pour une clé K de longueur $|K|$ et la i-ème lettre du message :

$$C_i = \left(P_i + K_{\,i \bmod |K|}\right) \bmod 26.$$

Le compteur de clé n'avance que sur les lettres : espaces et ponctuation ne consomment pas de clé.

3.2.2 Implémentation

```
def vigenere_encrypt(plaintext, key):
    result = []
    key = key.upper()
    key_index = 0
    for char in plaintext:
        if char in string.ascii_letters:
            shift = ord(key[key_index % len(key)]) - ord("A")
            base = ord("A") if char.isupper() else ord("a")
            result.append(chr((ord(char) - base + shift) % 26 + base))
            key_index += 1
        else:
```

```
            result.append(char)
    return "".join(result)
```

La fonction `vigenere_decrypt` est identique, avec `- shift` à la place de `+ shift`.

3.2.3 Résultats

Clé	Message	Chiffré
`crypto`	`ATTAQUE A L AUBE`	`CKRPJIG R J PNPG`
`crypto`	`AAAAAAAAAAAAAAAAA`	`CRYPTOCRYPTOCRYPTO`
`A`	`NHAR KBIR HADA ASSI SALMI`	identique au message
`CLEBEAUCOUPPLUSLONGUE`	`NHAR KBIR HADA ASSI SALMI`	`PSES OBCT VUSP LMKT GNRGM`
`key1` (avant correction)	`NHAR KBIR HADA ASSI SALMI`	`XLYB UFGB REBK KWQS CEJWS`
`key`	`NHAR KBIR HADA ASSI SALMI`	`XLYB OZSV FKHY KWQS WYVQG`

Table 4 – Tests du chiffre de Vigenère (le déchiffrement redonne toujours le message d'origine).

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Cle: crypto | Chiffre :CKRPJIG R J PNPG
Dechiffre: ATTAQUE A L AUBE
```

Figure 15 – Chiffrement et déchiffrement de `ATTAQUE A L AUBE` avec la clé `crypto`.

Observations.

- **Message monotone** : avec `AAAA...`, le chiffré est exactement la clé répétée ; la clé fuit si le message est régulier (figure ci-dessous).
- **Clé `A`** : décalage nul, le message n'est pas modifié. Une clé ne doit pas être composée uniquement de « A » (figure ci-dessous).
- **Clé longue** : le motif périodique disparaît visuellement, ce qui renforce la résistance à l'analyse de fréquences, idée à rapprocher du masque jetable (figure ci-dessous).

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Cle: crypto | Chiffre :CRYPTOCRYPTOCRYPTO
Dechiffre: AAAAAAAAAAAAAAAAA
```

Figure 16 – Un message composé de « A » révèle la clé `CRYPTO`.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Cle: A | Chiffre :NHAR KBIR HADA ASSI SALMI
Dechiffre: NHAR KBIR HADA ASSI SALMI

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>
```

Figure 17 – Avec la clé `A`, le chiffré est identique au message.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Cle: CLEBEAUCOUPPLUSLONGUE | Chiffre :PSES OBCT VUSP LMKT GNRGM
Dechiffre: NHAR KBIR HADA ASSI SALMI
```

Figure 18 – Clé `CLEBEAUCOUPPLUSLONGUE` : plus de répétition visible.

3.2.4 Bug découvert : la clé `key1`

Avec la clé `key1`, le programme ne lève aucune erreur et le déchiffrement fonctionne. Pourtant la clé est invalide :

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Cle: key1 | Chiffre :XLYB UFGB REBK KWQS CEJWS
Dechiffre: NHAR KBIR HADA ASSI SALMI
```

Figure 19 – La clé `key1` est acceptée sans erreur.

Analyse. Le caractère `1` a pour code ASCII 49 :

$$\text{ord}(\texttt{1}) - \text{ord}(\texttt{A}) = 49 - 65 = -16 \equiv 10 \pmod{26},$$

soit le décalage de la lettre `K`. La clé `key1` est donc équivalente à `keyk` : deux clés différentes donnent le même chiffré (vérifié : `N` + 10 = `X`, `H` + 4 = `L`, ...). Le programme acceptait donc silencieusement une clé erronée, ce qui est une faiblesse de sécurité.

Correction. Validation de la clé au début des deux fonctions (elle rejette aussi la clé vide, qui provoquerait une division par zéro dans `% len(key)`) :

```
    if not key.isalpha() or not key.isascii():
        raise ValueError("La cle doit contenir uniquement des lettres A-Z")
```

Après correction, la clé `key1` est rejetée par un `ValueError`, alors qu'une clé valide (par exemple `key`) fonctionne normalement.

```
def vigenere_encrypt(plaintext, key):
    result =[]
    if not key.isalpha() or not key.isascii():
        raise ValueError("La cle doit contenir uniquement des lettres A-Z")
    key = key.upper()
    key_index = 0
    for char in plaintext:
        if char in string.ascii_letters:
            shift = ord (key[key_index % len(key)]) - ord("A")
            base = ord("A") if char.isupper() else ord("a")
            result.append(chr((ord(char) - base + shift) % 26 + base))
            key_index += 1
        else:
            result.append(char)
    return "".join(result)

def vigenere_decrypt(ciphertext, key):
    result = []
    if not key.isalpha() or not key.isascii():
        raise ValueError("La cle doit contenir uniquement des lettres A-Z")
    key = key.upper()
    key_index = 0
```

Figure 20 – Validation ajoutée au début de `vigenere_encrypt` et `vigenere_decrypt`.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "C:\Users\HP\Desktop\New folder\cryptovault\src\classical.py", line 61, in <module>
    enc = vigenere_encrypt(msg, key)
  File "C:\Users\HP\Desktop\New folder\cryptovault\src\classical.py", line 22, in vigenere_encrypt
    raise ValueError("La cle doit contenir uniquement des lettres A-Z")
ValueError: La cle doit contenir uniquement des lettres A-Z
```

Figure 21 – La clé `key1` est maintenant refusée : `ValueError`.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.classical
Cle: key | Chiffre :XLYB OZSV FKHY KWQS WYVQG
Dechiffre: NHAR KBIR HADA ASSI SALMI
```

Figure 22 – Une clé valide (`key`) est acceptée et le déchiffrement est correct.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git add src/classical.py

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git commit -m "fix: validate vigenere key (lette
rs only, non-empty)"
[main a411fc9] fix: validate vigenere key (letters only, non-empty)
 1 file changed, 5 insertions(+), 1 deletion(-)

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 514 bytes | 257.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/secboy-star/cryptovault.git
   f14855c..a411fc9  main -> main

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>
```

Figure 23 – Commit de correction `fix: validate vigenere key (letters only, non-empty)` (hash `a411fc9`).

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git add src/classical.py

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git commit -m "feat: add vigenere cipher (encryp
t, decrypt"
[main f14855c] feat: add vigenere cipher (encrypt, decrypt
 1 file changed, 36 insertions(+), 1 deletion(-)

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 920 bytes | 306.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/secboy-star/cryptovault.git
   57dfdb3..f14855c  main -> main
```

Figure 24 – Commit initial de l'étape Vigenère (hash `f14855c`) publié sur GitHub.

3.2.5 Limites

Une clé courte et répétée reste attaquable (test de Kasiski, indice de coïncidence).

3.3 Étape 3 : Chiffrement XOR

3.3.1 Principe

L'opération « ou exclusif » (XOR, notée $\oplus$) travaille bit à bit. Sa propriété essentielle est

$$(a \oplus k) \oplus k = a,$$

donc la même fonction chiffre et déchiffre. C'est une brique de base d'AES, de ChaCha20 et du masque jetable.

3.3.2 Implémentation

```python
def xor_encrypt(data: bytes, key: bytes) -> bytes:
    return bytes([b ^ key[i % len(key)] for i, b in enumerate(data)])


xor_decrypt = xor_encrypt  # XOR est symetrique
```

3.3.3 Résultats

Message `Donnees sensibles`, clé `CLE_SECRETE`. Vérification manuelle des premiers octets :

Message	Clé	XOR (hexa)	Octet chiffré
`D` (0x44)	`C` (0x43)	$0x44 \oplus 0x43$	`07`
`o` (0x6f)	`L` (0x4c)	$0x6f \oplus 0x4c$	`23`
`n` (0x6e)	`E` (0x45)	$0x6e \oplus 0x45$	`2b`

Table 5 – Calcul manuel des trois premiers octets (conforme à la sortie du programme).

Le chiffré obtenu est `07232b313620307236312b302527333636`.

3.3.4 Attaque à texte clair connu

Si l'attaquant connaît un message et son chiffré, il retrouve la clé : $c \oplus p = (p \oplus k) \oplus p = k$. L'exécution de `xor_encrypt(encrypted, message)` renvoie `b'CLE_SECRETECLE_SE'` : la clé complète est révélée.

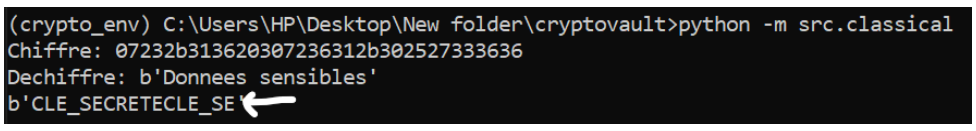


Figure 25 – Chiffré en hexadécimal, déchiffré, et clé retrouvée par texte clair connu.

3.3.5 Malléabilité : aucun contrôle d'intégrité

Un seul bit du chiffré est inversé (`tampered[0] ^= 0x01`). Le déchiffrement réussit sans erreur et renvoie un texte altéré : `b'Eonnees sensibles'` au lieu de `b'Donnees sensibles'`.

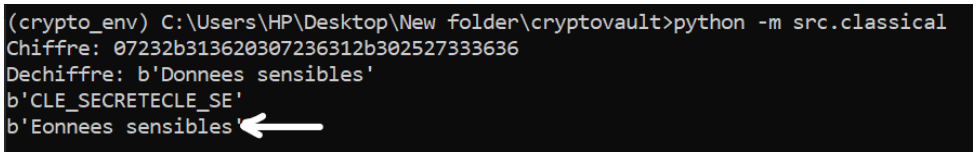


Figure 26 – Modification d'un bit : le texte est modifié sans qu'aucune erreur ne soit signalée.

3.3.6 Conclusion

Le XOR avec une clé répétée n'offre ni résistance au texte clair connu ni intégrité. Il n'est sûr que sous la forme du masque jetable : clé aléatoire, aussi longue que le message et utilisée une seule fois.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git add src/classical.py

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git commit -m "feat: add cipher with repeating k
ey"
[main f736ee1] feat: add cipher with repeating key
 1 file changed, 19 insertions(+), 1 deletion(-)

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 658 bytes | 219.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/secboy-star/cryptovault.git
   a411fc9..f736ee1  main -> main
```

Figure 27 – Commit de l'étape XOR publié sur GitHub.

3.4 Étape 4 : AES-256-GCM

3.4.1 Principe

AES est le standard de chiffrement symétrique du NIST. Le mode GCM (Galois/Counter Mode) assure **simultanément** :

— la **confidentialité** (le message est illisible sans la clé) ;

— l'**intégrité et l'authenticité** grâce à une étiquette (*tag*) : si le chiffré est modifié, le déchiffrement échoue.

Élément	Rôle
Clé (32 octets)	Secret partagé ; 32 octets = AES-256 (16 ou 24 octets = AES-128 ou 192)
Nonce	Valeur aléatoire différente à chaque chiffrement avec une même clé
Tag	Sceau d'authentification vérifié au déchiffrement

Table 6 – Éléments d'un chiffrement AES-GCM.

3.4.2 Implémentation (`src/symmetric.py`)

```python
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
import base64


def aes_gcm_encrypt(plaintext: str, key: bytes) -> dict:
    cipher = AES.new(key, AES.MODE_GCM)
    ciphertext, tag = cipher.encrypt_and_digest(plaintext.encode("utf-8"))
    return {
        "ciphertext": base64.b64encode(ciphertext).decode(),
        "nonce": base64.b64encode(cipher.nonce).decode(),
        "tag": base64.b64encode(tag).decode(),
    }


def aes_gcm_decrypt(enc_data: dict, key: bytes) -> str:
```

```
    cipher = AES.new(key, AES.MODE_GCM,
                     nonce=base64.b64decode(enc_data["nonce"]))
    return cipher.decrypt_and_verify(
        base64.b64decode(enc_data["ciphertext"]),
        base64.b64decode(enc_data["tag"])).decode("utf-8")
```

Base64 convertit les octets en texte afin de les afficher ou de les stocker (JSON, fichier). La clé est produite par `get_random_bytes(32)` (générateur aléatoire cryptographique), jamais par le module `random`.

3.4.3 Résultats

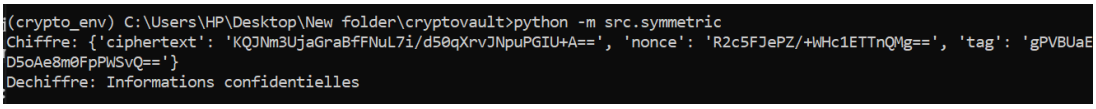


Figure 28 – Chiffrement puis déchiffrement de « Informations confidentielles » avec AES-256-GCM.

Non-déterminisme. Deux exécutions sur le même message produisent des chiffrés et des nonces différents (clé et nonce sont aléatoires), contrairement à César et XOR qui donnent toujours le même chiffré pour un même couple message/clé.

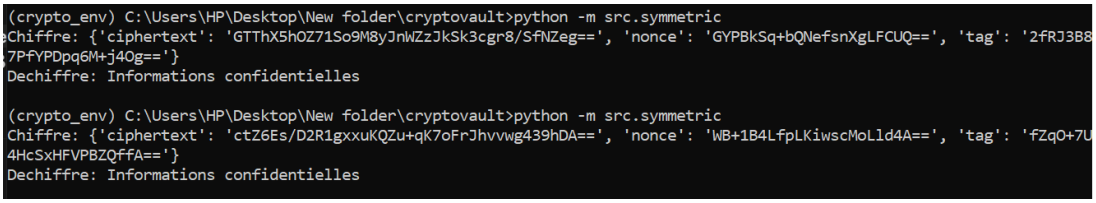


Figure 29 – Deux exécutions successives : chiffrés et nonces différents.

Détection d'une modification. Le même test que pour XOR (un bit inversé dans le chiffré) est appliqué à AES-GCM :

```
raw = bytearray(base64.b64decode(enc["ciphertext"]))
raw[0] ^= 0x01
enc["ciphertext"] = base64.b64encode(bytes(raw)).decode()
try:
    print(aes_gcm_decrypt(enc, key))
except ValueError as e:
    print("Attaque detectee:", e)
```

```
23
24  if __name__ == "__main__":
25      key = get_random_bytes(32)
26      msg = "Informations confidentielles"
27      enc = aes_gcm_encrypt(msg, key)
28      print(f"Chiffre: {enc}")
29      print(f"Dechiffre: {aes_gcm_decrypt(enc, key)}")
30
31      raw = bytearray(base64.b64decode(enc["ciphertext"]))
32      raw[0] ^= 0x01
33      enc["ciphertext"] = base64.b64encode(bytes(raw)).de
34      try:
35          print(aes_gcm_decrypt(enc, key))
36      except ValueError as e:
37          print("Attaque detectee:", e)
```

Figure 30 – Code du test de modification d'un bit sur le chiffré AES-GCM.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.symmetric
Chiffre: {'ciphertext': 'tFSrqW8/m2OQrsjFFf9TWCXbb4gmuoec2nnFAw==', 'nonce': 'Vo7t+p
XnR4zqQc/pAD2TFg==', 'tag': 'I+idDbDuVy0/njq/ZB4t+Q=='}
Dechiffre: Informations confidentielles
Attaque detectee: MAC check failed
```

Figure 31 – Avec AES-GCM, la modification d'un seul bit est détectée (`MAC check failed`).

Autres essais.

- Clé incorrecte au déchiffrement : `MAC check failed`
- Clé de 10 octets : `Incorrect AES key length (10 bytes)`
- Tailles : nonce = 16 octets, tag = 16 octets

Ces essais sont exécutés par le bloc `__main__` de `src/symmetric.py` :

```
if __name__ == "__main__":
    key = get_random_bytes(32)
    msg = "Informations confidentielles"
    enc2 = aes_gcm_encrypt(msg, key)

    try:
        print(aes_gcm_decrypt(enc2, get_random_bytes(32)))
    except ValueError as e:
        print("Mauvaise cle:", e)

    try:
        aes_gcm_encrypt(msg, get_random_bytes(10))
    except ValueError as e:
        print("Cle de 10 octets:", e)

    print("nonce =", len(base64.b64decode(enc2["nonce"])), "octets")
    print("tag   =", len(base64.b64decode(enc2["tag"])), "octets")
    '''
```

Figure 32 – Code des essais complémentaires : mauvaise clé, clé de 10 octets, tailles du nonce et du tag.

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>python -m src.symmetric
Mauvaise cle: MAC check failed
Cle de 10 octets: Incorrect AES key length (10 bytes)
nonce = 16 octets
tag   = 16 octets
```

Figure 33 – Résultats : `MAC check failed`, clé de 10 octets refusée, nonce et tag de 16 octets.

3.4.4 Comparaison XOR / AES-GCM

Critère	XOR (clé répétée)	AES-256-GCM
Même message, même clé	Toujours le même chiffré	Chiffré différent (nonce)
Texte clair connu	La clé est retrouvée	Clé non récupérable
Un bit modifié	Non détecté (`Eonnees`)	`MAC check failed`
Intégrité	Aucune	Tag d'authentification

Table 7 – Comparaison des deux mécanismes sur les mêmes essais.

3.4.5 Bonnes pratiques

— Ne jamais réutiliser un nonce avec la même clé : en GCM, cela compromet la confidentialité et permet de falsifier des messages. La bibliothèque génère un nonce aléatoire à chaque appel de `AES.new`.

— Générer les clés avec `get_random_bytes`.

— Ne jamais écrire une clé en dur dans le code (thème du gestionnaire de secrets, niveau 3).

```
(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git add src/symmetric.py

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git commit -m "feat: add AES-256-GCM encryption with integrity c
heck"
[main 3126729] feat: add AES-256-GCM encryption with integrity check
 1 file changed, 37 insertions(+)

(crypto_env) C:\Users\HP\Desktop\New folder\cryptovault>git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 844 bytes | 211.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/secboy-star/cryptovault.git
   9573ba3..3126729  main -> main
```

Figure 34 – Commit de l'étape AES-256-GCM publié sur GitHub.

4 Historique des commits

Commit	Contenu
`3784701`	Structure du projet et dépendances
`b19ca29`	Ajout du chiffre de César (chiffrement, déchiffrement, force brute)
`6bae3c5`	Documentation : section César du compte rendu et captures
`57dfdb3`	Correction de César (lettres ASCII uniquement)
`f14855c`	Ajout du chiffre de Vigenère
`a411fc9`	Validation de la clé de Vigenère (lettres uniquement, non vide)
`f736ee1`	Ajout du chiffrement XOR
`3126729`	Ajout d'AES-256-GCM avec vérification d'intégrité

Table 8 – Principaux commits publiés sur `main` (dépôt `https://github.com/secboy-star/cryptovault`).

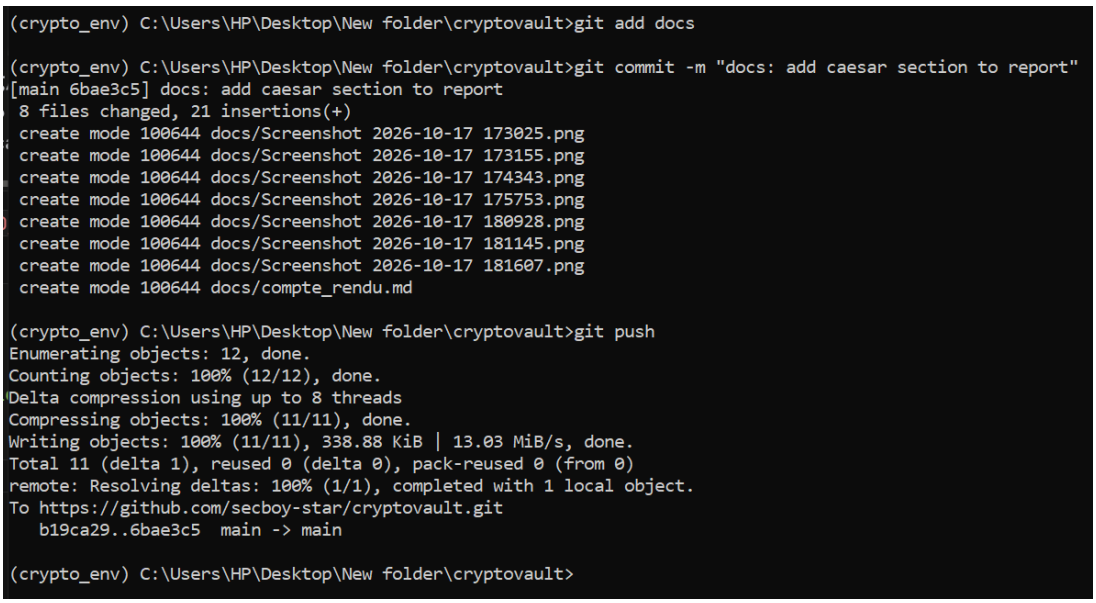


Figure 35 – Publication de la documentation (compte rendu et captures) sur GitHub, hash `6bae3c5`.

5 Difficultés rencontrées et leçons

Problème	Cause	Leçon
« ö » mal déchiffré (César)	`isalpha()` accepte l'Unicode, la formule suppose A–Z (chiffrement non injectif)	Un chiffrement doit être réversible ; restreindre explicitement l'alphabet
Clé `key1` acceptée (Vigenère)	`ord('1') - ord('A')` $= -16 \equiv 10 \pmod{26}$	Valider les entrées : une clé invalide ne doit pas fonctionner en silence
Modification non détectée (XOR)	Aucun contrôle d'intégrité	Utiliser un mode authentifié (AES-GCM)

Table 9 – Synthèse des erreurs et de ce qu'elles ont appris.

6 Limites et améliorations prévues

— **César et Vigenère** : algorithmes pédagogiques, non sûrs ; la gestion des lettres accentuées pourrait passer par une normalisation Unicode.

— **XOR** : sûr uniquement comme masque jetable ; ici démonstration de ses faiblesses.

— **AES-GCM** : les clés sont générées en mémoire et non stockées ; la gestion et la rotation des clés seront traitées avec le gestionnaire de secrets (niveau 3).

— **Tests** : les essais sont pour l'instant des scripts dans `__main__` ; ils seront remplacés par des tests unitaires (`unittest`) dans le dossier `tests/` au niveau 1.

7 Conclusion et suite

Ce premier volet a permis de passer des chiffrements classiques, instructifs mais faibles, à un chiffrement authentifié moderne, et de comprendre *pourquoi* chaque étape succède à la précédente (attaque par force brute, fuite de clé, malléabilité). Les erreurs rencontrées ont été la meilleure source d'apprentissage.

Suite du projet. RSA (chiffrement OAEP et signature), échange de clés Diffie-Hellman, puis le niveau 1 (module de hachage et tests unitaires), le niveau 2 (PKI X.509 et messagerie E2E) et le niveau 3 (gestionnaire de secrets avec rotation et audit).

Références

— Documentation PyCryptodome : `https://pycryptodome.readthedocs.io/`

— Documentation `cryptography` : `https://cryptography.io/en/latest/`

— NIST SP 800-38D : *Recommendation for Block Cipher Modes of Operation : Galois/Counter Mode (GCM)*

— NIST SP 800-57 : *Recommendation for Key Management*

— Dépôt du projet : `https://github.com/secboy-star/cryptovault`